

PATROL: An Evolutionary Approach to Automatic Test Pattern Generation for Hardware Trojan Detection Leveraging PSO-GA Hybrid Techniques.

Mostafa Hosseini¹, Ali Azarpeyvand^{1,2}, and Tara Ghasempouri²

¹University of Zanjan, Zanjan, Iran

²Tallinn University of Technology, Tallinn, Estonia

Abstract—The global distribution of the semiconductor supply chain has heightened the risk of hardware Trojans (HTs), small malicious circuits that adversaries may embed during various stages of the system-on-chip (SoC) design process. Frequently implanted by untrusted third parties, these HTs can operate covertly and, when activated, pose a serious threat to the integrity, performance, and functionality of the system. Although there are promising test generation techniques for HT detection, they face two significant practical limitations: a lack of scalability for large designs and insufficient trigger coverage. The effective detection of HTs requires the application of appropriate test vectors. This paper introduces PATROL, a novel algorithm that combines Particle Swarm Optimization (PSO) and Genetic Algorithms (GA) within a scalable framework for the detection of HTs. This framework employs Automated Test Pattern Generation (ATPG)-based activation to achieve high trigger coverage. This approach significantly accelerates the convergence towards a solution while substantially improving the solution accuracy. Our experimental results demonstrate that our proposed method is more than 43 times faster and achieves an average increase in trigger coverage of more than 34%, significantly outperforming state-of-the-art test generation techniques for Trojan detection.

Index Terms—Hardware Trojan, Test Generation, Particle Swarm Optimization, Genetic Algorithm, Hybrid Metaheuristic.

I. INTRODUCTION

Modern chip manufacturing involves multiple steps, each managed by different companies. Outsourcing Integrated Circuit (IC) fabrication (from design to testing) can expedite production and reduce costs. However, it also opens opportunities for malicious actors to embed concealed vulnerabilities, known as HTs, into chips [1]. An HT refers to any unauthorized alteration in the design of a chip with the intent of achieving or facilitating the achievement of a malicious objective. Reliance on third-party Intellectual Property (IP) as trusted modules presents a potential vulnerability that can be life-threatening [2]. Developing an IC involves four distinct phases: specification, behavioral modeling, gate-level design, and physical layout. Gate-level design is a mandatory step in the creation of any IC [3]. Third-party IPs at the behavior level are synthesized to gate-level descriptions, and layout-level synthesis includes location information. Due to the importance of gate-level netlists, this paper focuses on HT detection in these netlists.

There are a variety of methods to detect HTs. Recent approaches frequently involve generating test patterns based on golden reference models, such as Transaction-Level Models (TLM) or Register Transfer Level (RTL) models, and applying these patterns to gate-level implementations (assuming HTs are introduced during synthesis) or to integrated circuits (assuming HTs are introduced during fabrication) [4]. Although these methods show promise in enhancing the probability of activating elusive and concealed triggers, they encounter considerable challenges related to scalability and trigger coverage [5].

In this article, we introduce PATROL, an efficient heuristic algorithm for HT detection through test generation. PATROL exhibits faster convergence, independent of the initial population, and delivers more precise results. This work marks the first application of a hybrid algorithm for HT detection. The principal contributions of this work are as follows:

- **Development of PATROL a PSO-GA hybrid algorithm:** We propose a novel hybrid algorithm, PATROL, that integrates PSO with GA to enhance ATPG for HT detection.
- **Scalability and extensive trigger coverage:** The quick convergence of PSO enables the PATROL to identify effective solutions more rapidly. Meanwhile, GA's crossover and mutation techniques assist in avoiding local optima where PSO could become trapped.
- **Effectiveness:** The effectiveness of the PATROL is evaluated across eight different ISCAS-85 benchmark circuits [6]. Performance is compared with traditional logic testing methods, which confirms the validity of the suggested approach.

The remainder of the paper is organized as follows: Section II reviews previous related works. Section III introduces some preliminary definitions. Section IV provides an overview of the proposed method. Section V reports the experimental results. Finally, Section VI concludes the paper.

II. RELATED WORK

Simulation is the predominant method for validation, employing both random and constrained random tests. Although random tests effectively encompass a wide range of easily detectable scenarios, they fail to adequately address complex

corner cases and difficult-to-detect scenarios in a reasonable time frame [7]. Directed tests are promising for covering these remaining scenarios. However, manual creation of directed tests is both time-consuming and prone to errors. Recent advancements seek to mitigate this issue through the development of automated test generation techniques [8].

Jayasena and Mishra [4] introduced an ATPG-based framework for generating test cases that target rare events in HTs. Unlike traditional bit-flipping methods, their approach used ATPG tools to create new test vectors, ensuring the tests' validity and efficiency in activating infrequent events. Their work demonstrated the effectiveness of ATPG-based N-activation and maximal clique activation in stimulating rare events in complex hardware environments.

Chakraborty et al. [9] proposed MERO, a statistical method based on the N-detect principle to activate HT mechanisms by triggering signals. By statistically activating each trigger signal N times, they aimed to increase the chances of detecting and activating potential Trojans through the generation of specific test vectors. The primary limitations of using MERO in this context are as follows: Similar to methods like FANCI [10] and VeriTrust [11], MERO encounters significant challenges due to its high time complexity. These approaches can become impractical for circuits with a large number of inputs, as their time complexity exceeds $O(2^n)$, where n represents the number of circuit inputs. Moreover, MERO may have difficulty detecting nets with low-switching activity, which are often employed as triggers for HTs. This is because MERO primarily relies on signal probability analysis, which may not effectively capture these low-activity nets, potentially allowing them to escape detection.

Nourian et al. [12] presented a novel methodology for the detection of HT that synergistically combines GA with principles of logic testing. An initial test vector was meticulously constructed and subsequently evaluated using a fitness function that encompasses netlist specifications, GA parameters, and a predefined threshold. The main limitations of employing the GA in this context are as follows: the search process inherent to the algorithm is unguided and lacks direction. Furthermore, this algorithm shows a high sensitivity to both the initial population parameters and the initial parameter configurations. Moreover, the proposed approach, which is based on GA, requires significantly more time to generate tests compared to alternative methods and is also computationally intensive.

S. Chakraborty et al. [13] introduced a novel test pattern generation method using PSO for detecting HTs. PSO, an evolutionary computation technique, involves each particle representing a potential solution. These particles adjust their positions in the search space and update their velocities based on their own experience and that of their neighbors, enabling them to find optimal positions for improved solutions. Similar to GA, PSO is sensitive to the initial population. One of the key characteristics of PSO is its rapid convergence towards the global optimum in the early stages of the search, followed by a slower convergence as it approaches the optimum. PSO demonstrates significant strength in local search, as particles

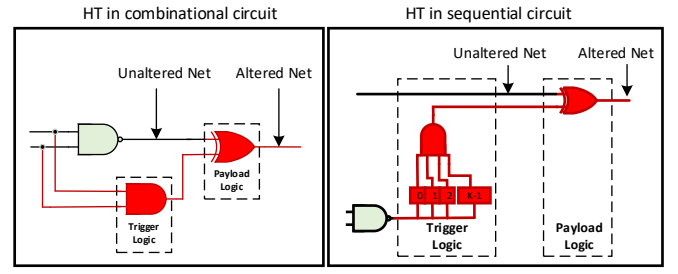


Fig. 1: HTs in both combinational and sequential circuits.

tend to converge towards the most promising positions identified so far. However, it may exhibit limitations in effectively conducting a global search [14].

III. PRELIMINARIES

In this section, we shall concisely elucidate the definitions and concepts employed within this article to facilitate the readers' comprehension of the proposed approach.

Definition 1: Hardware Trojan (HT) circuits are extremely small and difficult to detect. An HT consists of a payload and, optionally, a trigger mechanism. The trigger monitors the base circuit for specific conditions, and once these conditions are met, it activates the payload. The payload is the malicious component responsible for carrying out the harmful actions of the HT. HTs without triggers generally function as always-on Trojans. Digital HT triggers can be designed using either combinational or sequential circuits. Combinational triggers activate when a specific set of signals is received, while sequential triggers depend on a particular sequence of signals or events, such as a counter reaching a certain value or a repeating signal pattern over time. Figure 1 illustrates a visual example of an HT.

Definition 2: Switching Activity is the key factor we focused on to identify a group of potentially suspicious nets within a gate-level design. The most significant characteristic of an HT is its extremely infrequent activity over time. This implies that the trigger nets of the HT in the design exhibit very low activity. A low-activity net, Net_i has a switching rate (α) significantly lower than that of other nets in the circuit. Switching Activity can be calculated using Equation 1:

$$\alpha = \frac{\# \text{ of Transitions } (Net_i)}{2^n} \quad (1)$$

In this context, n is the number of circuit inputs, 2^n represents the total number of input patterns, and the number of transitions of Net_i ($0 \rightarrow 1$ or $1 \rightarrow 0$) across all 2^n applied input patterns. It is understood that a net with extremely low activity typically maintains a constant logic state, either '1' or '0'. To identify these nets, we simulate circuit behavior with a moderate number of random test vectors (e.g., 1000) and then isolate nets whose rare value occurrences meet a specific threshold [15].

Definition 3: Fitness Function is used to determine the optimal solutions in each iteration of the PSO-GA hybrid algorithm by calculating and comparing the fitness of each

generated test vector. For combinational circuits, fitness is defined as the number of switching activities detected by the test vector.

To compute the fitness function in PATROL, we begin by initializing the fitness value to zero. This process will accumulate the fitness score by examining each gate in the netlist. The function iterates over every gate present in the netlist and checks if the current gate is classified as a rare gate. If both conditions (that the gate is rare and its output is the rare value) are satisfied, the fitness value is increased by one. Finally, the computed fitness value is returned.

Definition 4: Hybrid Algorithm combines multiple optimization or computational techniques to leverage the strengths and mitigate the weaknesses of the individual methods. The PSO-GA hybrid algorithm combines the PSO and GA techniques. Initially, PSO creates a population close to the global optimum, setting the stage for GA. Then, the algorithm shifts to GA, which uses this initial population to further refine the optimization process.

Definition 5: Circuit Representation uses directed graphs (DG = V, E), where V represents the set of vertices and E denotes the set of edges. In this model, logic gates are represented as vertices, while wires (or nets) are depicted as edges. Logic gates are modeled as vertices, while wires (nets) are represented by edges. Subsequently, we shall levelize the circuit. The Primary Inputs (PIs) of the circuit are regarded as level 0. The output level of each gate is calculated according to equation 2. The circuit instances utilized in this study are derived from the ISCAS-85 benchmark circuits.

$$\text{Level}(\text{output}) = \max(\text{Level}(\text{in1}), \text{Level}(\text{in2})) + 1 \quad (2)$$

IV. METHODOLOGY

Current logic testing-based Trojan detection methods face two key challenges: high computational complexity, resulting in prolonged test generation times, and low detection accuracy, leading to inadequate trigger coverage [16].

This paper introduces PATROL, a pioneering heuristic algorithm that integrates PSO and GA to refine traditional optimization methods for automatic test pattern generation, with a specific focus on HT detection. The PSO algorithm initially generates a test vector with high trigger coverage by exploring large areas of the solution space and quickly converging toward promising regions. While PSO efficiently identifies good candidate solutions, it often struggles to maintain diversity, leading to premature convergence in local optima. To overcome this, the algorithm transitions to GA, which takes over after PSO has found promising solutions. GA introduces genetic diversity through crossover and mutation, enabling more precise local optimization and refinement of the solutions. This combined approach of broad exploration with PSO followed by GA's fine-tuning enhances the overall effectiveness of the optimization process for HT detection. PATROL operates in two phases:

1. **Initialization:** The PSO algorithm is utilized to generate an initial population of test vectors for combinational circuits,

with these vectors meticulously chosen to approximate the global optimum. Algorithm 1 outlines a PSO method for generating initial test patterns.

Algorithm 1 PSO based test generation

```

1: Start
2:  $Num\_Particles \leftarrow 50$ 
3:  $Max\_Iterations \leftarrow 100$ 
4:  $Num\_Variables \leftarrow \text{length PIs for each CKTs}$ 
5:  $Inertia\_Weight \leftarrow 0.7$ 
6:  $Cognitive\_Weight \leftarrow 1.1$ 
7:  $Social\_Weight \leftarrow 1.1$ 
8: Zero Initialize  $Swarm\_position$ ,  $fitness\_values$  arrays
9: Generate random test patterns and evaluate fitness
10: Initialize  $swarm\_velocity$ ,  $personal\_best\_position$ ,  $personal\_best\_fitness$ ,  $global\_best\_index$ ,  $global\_best\_position$ ,  $global\_best\_fitness$ 
11: Initialize an empty list  $best\_individuals$ 
12: for  $iteration = 1, 2, \dots, max\_iterations$  do
13:   Update velocities and positions
14:   Evaluate fitness values for new positions
15:   Update position and fitness values of personal best and global best individuals
16:   Add the top 2 individuals to  $best\_individuals$  and update the  $best\_individuals$ 
17: end for
18: Sort  $best\_individuals$  based on fitness values
19: return  $best\_individuals$ 
20: End

```

The process begins with initializing the PSO parameters and swarm of particles, each representing a potential solution, with their positions, velocities, and fitness values (Lines 1-8). The primary objective is to generate random test patterns to assess fitness (Line 9). Additional parameters such as the velocity of the swarm, the global best index, the global best position, the personal best position, the personal best fitness, and the global best fitness are also necessary (Line 10). The algorithm executes for a predetermined number of iterations or until a specified termination condition is met, as shown in (line 12). Particles can update their positions (Line 13) by equation 3:

$$x_i^{(t+1)} = x_i^t + v_i^t \cdot t \quad (3)$$

Where X_i represents a randomly initialized position, and V_i represents a randomly initialized velocity. The velocity of the random particle is determined by the formula outlined in equation 4.

$$v_{k+1}^i = w \cdot v_k^i + c_1 \cdot r_1 \cdot (x_{best}^t - x_i^t) + c_2 \cdot r_2 \cdot (g_{best}^t - x_i^t) \quad (4)$$

Here, x_{best} represents the best particle position, g_{best} denotes the best global position, w is the inertia weight, c_1 and c_2 are constants, and r_1 and r_2 are random parameters within the range [0, 1]. Since r_1 and r_2 must fall within the range [0, 1], we use the sigmoid function. This function is applied to adjust the velocity data, as indicated by the following statement. In PSO, the sigmoid transformation maps velocity data to a restricted range between 0 and 1, enabling non-linear adjustments that balance exploration and exploitation

according to equation 5. For binary conversion of fractional values, the rand function from the numpy library is used, applying a 0.5 threshold to assign values of 0 or 1. Elements below 0.5 are set to 0, while those at or above 0.5 are set to 1.

$$\frac{1}{1 + e^{-\text{swarm_velocity}[\text{particle}]}} \quad (5)$$

After velocities are updated, particles move to new positions and evaluate their fitness value (Line 14). The fitness value starts at zero, and rare gates are checked for their unique outputs. If these outputs match the rarity criteria, the fitness value increases. Update the particle's position if this new fitness surpasses the personal best (Line 15). By comparing the personal best positions of all the particles in the swarm and selecting the one with the highest fitness value, we can identify the global best position. The list called "best individuals" is now sorted by fitness values, allowing us to identify the top individuals (Lines 18-19).

2. **Refinement:** GA takes over the optimization process, using the population generated by PSO as a starting point. GA continues to evolve the population until a satisfactory solution is found. Algorithm 2 provides a detailed method for generating test vectors with this GA.

Algorithm 2 GA for test vector generation

```

1: Input: gbest, MR
2: num_test_vectors  $\leftarrow$  100
3: MR  $\leftarrow$  0.001
4: test_vectors  $\leftarrow$  [gbest, copy(gbest)]
5: while len(test_vectors)  $\neq$  num_test_vectors do
6:   parent_indices  $\leftarrow$  Random unique indices
     range [0, len(test_vectors) - 1].
7:   parent1  $\leftarrow$  test_vectors[parent_indices[0]]
8:   parent2  $\leftarrow$  test_vectors[parent_indices[1]]
9:   child1, child2  $\leftarrow$  Crossover(parent1, parent2)
10:  test_vectors.append(Mutate(child1, MR))
11:  if len(test_vectors)  $\neq$  num_test_vectors then
12:    test_vectors.append(Mutate(child2, MR))
13:  end if
14: end while
15: Evaluate fitness of test_vectors
16: test_vectors_fitness  $\leftarrow$  [fitness_function(tv)
    for tv in test_vectors]
17: sorted_indices  $\leftarrow$  indices that would sort
    test_vectors_fitness in descending
    order
18: top_test_vectors  $\leftarrow$ 
    [test_vectors[i] for i in sorted_indices[: 100]]
19: return top_test_vectors

```

In the GA, use the best population generated by the PSO as the initial population for the GA. Assign the value 100 to *Num_test_vectors* and 0.001 to Mutation Rate (MR), respectively (Lines 1-4). While the *test_vectors* list contains fewer than *num_test_vectors*, it is expanded until it reaches *num_test_vectors*. To do this, two indices are randomly selected from the existing *test_vectors*, which correspond to two "parent" vectors (Lines 6-9). A crossover operation is then applied to these parents, combining their "genes" (values in the vector) to create two "child" vectors according to Algorithm 3

(Line 9). The first child (*child1*) undergoes mutation as mentioned in Algorithm 4, introducing small random changes to its values, and is then added to the *test_vectors* list (line 10). If the list still has fewer than 100 vectors, the second child (*child2*) is also mutated and added (Lines 11-12). After all vectors are generated, the algorithm calculates a fitness value for each vector and sorts the list in descending order of fitness. Only the top 100 vectors, based on their fitness, are retained for further use or as the final result.

Algorithm 3 performs a crossover operation, combining parts of two parent vectors (*parent1* and *parent2*) to create two offspring. This aims to produce new solutions with the best traits of both parents. First, a random crossover point is selected within the range [1, len(*parent1*) - 1], ensuring that it does not occur at the edges to avoid duplicate offspring. *Child1* is created by merging the initial segment of *parent1* with the latter segment of *parent2*, while *child2* is formed by combining the start of *parent2* with the remainder of *parent1*. The algorithm returns *child1* and *child2* as the result of the crossover.

Algorithm 3 Crossover operation

```

1: Input: parent1, parent2
2: crossover_point  $\leftarrow$  random integer in the range
   [1, len(parent1) - 1]
3: child1  $\leftarrow$  concatenate(parent1[:
   crossover_point], parent2[crossover_point :])
4: child2  $\leftarrow$  concatenate(parent2[:
   crossover_point], parent1[crossover_point :])
5: return child1, child2

```

Algorithm 4 implements a mutation operation, which introduces minor random modifications to a child (individual) within a GA. This process helps maintain diversity in the population by preventing premature convergence to suboptimal solutions.

The algorithm takes two inputs: a test vector—a sequence of bits generated by Algorithm 3—and a *mutation_rate*, representing the probability that each bit will be flipped. It iterates through each bit in the sequence, from index 0 to len(*child*) - 1. For each bit, a random number between 0 and 1 is generated. If this number is less than *mutation_rate*, the bit is flipped (0 becomes 1, and 1 becomes 0). After processing all bits, the algorithm returns the mutated child sequence.

Algorithm 4 Mutation operation

```

1: Input: child, mutation_rate
2: for i = 0 to len(child) - 1 do
3:   if random number < mutation_rate then
4:     child[i]  $\leftarrow$  Flip Bit(child[i])
5:   end if
6: end for
7: return child

```

V. EXPERIMENTAL RESULTS

In this section, we present the results of applying PATROL to the ISCAS-85 benchmarks [6]. All experiments were

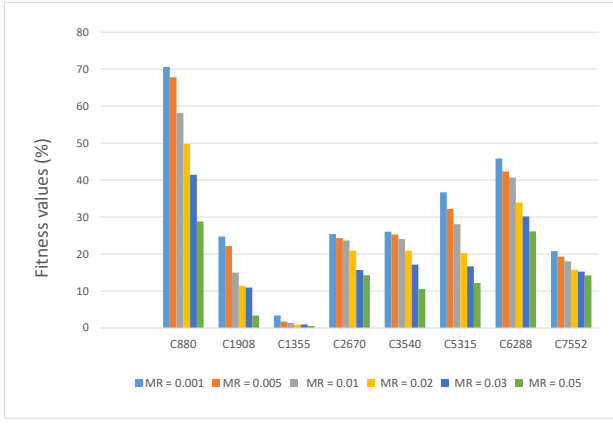


Fig. 2: Impact of MR values on average coverage of 100 test vectors.

conducted using a system with a 7th Gen Intel® Core™ processor @ 2.80GHz with 4 cores, 16GB of RAM, and a 64-bit Windows 10 operating system.

PATROL reveals that modifying the values of infrequently activated network nodes, or rare nets, can be accomplished with a relatively small set of input stimuli. Since random test cases are generally inadequate for detecting vulnerabilities in highly complex circuits, targeting these rare nets is crucial for mitigating potential risks. To identify rare nets in our experiment, we applied two specific activations α range (α_{Range}) below 0.1 and between 0.1 and 0.2.

The MR helps prevent the algorithm from getting stuck in local optima. We found that as MR increases, the average fitness value of each test vector decreases. This is because a lower MR results in test vectors with minimal deviation from the initial vector, leading to a smaller Hamming distance and greater efficiency in triggering switching activity within the circuit. A very high MR, however, may slow the algorithm's convergence or prevent it from reaching an optimal solution. Figure 2 shows the effect of different MR values on the average fitness of 100 test vectors.

For a circuit with k inputs, there exist a total of 2^k possible input combinations. Table I presents the total time, measured in seconds, required to generate 100 test vectors for each circuit within the ISCAS-85 benchmark (CKTs), to achieve the desired coverage.

Improvement in runtime: Our proposed PSO-GA based algorithm, PATROL, frequently executes faster than PSO-based method in [13], GA-based methods, such as TRIAGE [12], or bit perturbation-based methods, like MERO [9]. PSO is highly effective in local search by guiding particles toward the most optimal known positions. However, it may exhibit weaknesses in global search. Conversely, GA demonstrates significant prowess in global search, effectively discovering new and diverse regions of the search space through the mechanisms of selection, crossover, and mutation. The integration of these two algorithms in the PATROL leverages the local search strengths of PSO while simultaneously capitalizing on the global search capabilities of GA. Table II shows that our proposed method

TABLE I: Execution time for various methods measured in seconds.

CKTs	α_{Range}	Execution time (second)			
		MERO	GA	PSO	PATROL
C880	(0.1, 0]	126.31	687.9	698.06	13.4
	(0.2, 0.1]	125.36	651.08	706.45	14.98
C1908	(0.1, 0]	145.34	1343.73	1309.07	25.72
	(0.2, 0.1]	126.7	1177.74	1185.42	22.83
C1355	(0.1, 0]	127.66	850.63	1104.74	20
	(0.2, 0.1]	108.7	792.18	837.57	15.17
C2670	(0.1, 0]	2610.63	3403.47	4096.19	66.29
	(0.2, 0.1]	2552.79	3443.33	3515.75	61.07
C3540	(0.1, 0]	400.1	2610.82	2532.92	46.33
	(0.2, 0.1]	411.13	2590.87	2615.46	49.44
C5315	(0.1, 0]	2901.98	5033.83	5201.7	93.61
	(0.2, 0.1]	3044.71	5567.19	5435.05	101.08
C6288	(0.1, 0]	359.49	3555.95	3562.69	62.92
	(0.2, 0.1]	382.76	3578.92	3675.63	78.22
C7552	(0.1, 0]	5493.49	8772.53	8373.92	150.11
	(0.2, 0.1]	5760.02	8462.89	8528.15	173.94

has achieved a significant increase in speed compared to other methods.

PATROL demonstrates a significant improvement in computational speed, outperforming the MERO, GA and PSO methods by factors of over 16, 50 and 52 times, respectively when generating 100 test vectors. In comparison to the MERO method, our approach is approximately 40 times faster in the optimal case for the C2670 circuit and exhibits a minimum speedup of 4 times for the C1908 circuit. When compared to the GA method, our method achieves a best-case acceleration of approximately 57 times for the C7552 circuit and a minimum acceleration of 41 times for the C1355 circuit. Relative to the PSO method, our approach is roughly 60 times faster in the best case for the C2670 circuit and at least 45 times faster for the C6288 circuit.

Improvement in trigger coverage: Our proposed method demonstrates a significant improvement in trigger coverage, surpassing the MERO and GA methods by more than 48% and 35%, respectively, when generating 100 test vectors. Additionally, it achieves nearly the same performance as the PSO method, with only a 1% difference. Figure 3 illustrates the average coverage for ISCAS-85 circuits. Specifically, compared to the MERO method, our approach achieves an

TABLE II: Improvement in terms of time.

CKTs	α_{Range}	Times performance		
		MERO	GA	PSO
C880	(0.1, 0]	8.4261	50.3358	51.0940
	(0.2, 0.1]	7.3685	42.4633	46.1595
C1908	(0.1, 0]	4.6509	51.2446	49.8970
	(0.2, 0.1]	4.5497	50.5874	50.9238
C1355	(0.1, 0]	5.3830	41.5315	54.2370
	(0.2, 0.1]	6.1655	51.2202	54.2123
C2670	(0.1, 0]	38.3820	50.3421	60.7920
	(0.2, 0.1]	40.8010	55.3833	56.5692
C3540	(0.1, 0]	7.6359	55.3527	53.6713
	(0.2, 0.1]	7.3157	51.4043	51.9017
C5315	(0.1, 0]	30.0007	52.7745	54.5678
	(0.2, 0.1]	29.1218	54.0771	52.7698
C6288	(0.1, 0]	4.7134	55.5154	55.6225
	(0.2, 0.1]	3.8934	44.7545	45.9909
C7552	(0.1, 0]	35.5964	57.4407	54.7852
	(0.2, 0.1]	32.1150	47.6541	48.0293

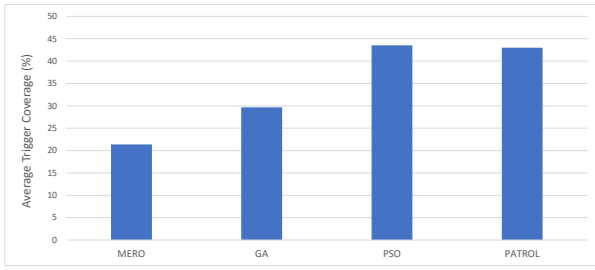


Fig. 3: Average coverage for ISCAS-85 circuits.

approximate 64% improvement in optimal coverage for the C6288 circuit, with a minimum coverage enhancement of 13% for the C1355 circuit. When compared to the GA method, our method delivers a best-case coverage improvement of approximately 75% for the C6288 circuit and a minimum coverage increase of 12% for the C1355 circuit. Relative to the PSO method, our approach achieves around a 6% improvement in the best-case coverage for the C880 circuit and at the worst case a -5% decrease in trigger coverage for the C5315 circuit.

TABLE III: Coverage performance for various circuit instances.

CKTs	α_{Range}	Total Rare Gate	Coverage Performance (%)			
			MERO	GA	PSO	PATROL
C880	(0.1, 0]	48	29.79	29.16	32.6	34.98
	(0.2, 0.1]	28	30.37	75	81.48	85.57
C1908	(0.1, 0]	104	12.4	11.53	25.96	25.73
	(0.2, 0.1]	9	43.33	55.55	77.77	77.33
C1355	(0.1, 0]	105	2.9	3.8	3.8	3.39
	(0.2, 0.1]	9	22.22	44.44	44.44	42.44
C2670	(0.1, 0]	78	17.3	8.9	25.64	25.41
	(0.2, 0.1]	121	30.74	44.35	55.37	54.52
C3540	(0.1, 0]	257	15.23	22.17	61	59.87
	(0.2, 0.1]	222	20.9	26.33	27.92	27.81
C5315	(0.1, 0]	46	17.52	17.02	39.13	36.69
	(0.2, 0.1]	155	28.06	34.14	56.12	50.9
C6288	(0.1, 0]	68	16.17	11.76	45.5	45.13
	(0.2, 0.1]	195	25.12	47.64	61.53	61.82
C7552	(0.1, 0]	155	13.16	19.35	23.87	23.9
	(0.2, 0.1]	143	16.64	23.77	34.26	32.62

VI. CONCLUSIONS

This research introduces PATROL, an advanced algorithm that integrates PSO with GA for the automated generation of test vectors aimed at detecting HTs. The primary objective of PATROL is to address the limitations inherent in each individual algorithm, thereby enhancing overall performance and offering a more effective solution for complex optimization problems. The efficacy of the proposed method was substantiated through a comparative analysis with the MERO, GA and PSO algorithms. The experimental findings indicate a substantial reduction in test generation time, achieving improvements of approximately over 16, 50, and 52 times, respectively. Additionally, there was a significant improvement in trigger coverage, surpassing the MERO and GA methods by over 48% and 35%, respectively. On average, our proposed

method achieves 1% lower trigger coverage compared to the PSO method, while being considerably faster. The findings reveal that the HA not only outperforms the base algorithms but also delivers superior solutions in a reduced timeframe. Furthermore, this algorithm exhibits diminished sensitivity to initial population values, which contributes to its increased stability and reliability in producing consistent results.

ACKNOWLEDGMENT

This work was supported in part by the Estonian Research Council grants PSG837.

REFERENCES

- [1] H. Zhang, Y. Zhou, and Y. Li, "A portable hardware trojan detection using graph attention networks," in *2023 IEEE 32nd Asian Test Symposium (ATS)*. IEEE, 2023, pp. 1–6.
- [2] Y. Rozhin, C. Luke, Y. Shih-Yuan, and F. M. A. Al, "Hardware trojan detection using graph neural networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022.
- [3] M. M. Rahimifar, H. Jahanirad, and M. Fathi, "Deep transfer learning approach for digital circuits vulnerability analysis," *Expert Systems with Applications*, vol. 237, p. 121757, 2024.
- [4] A. Jayasena and P. Mishra, "Scalable detection of hardware trojans using atpg-based activation of rare events," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.
- [5] M. R. Heidari Iman, S. Ahmadi-Pour, R. Drechsler, and T. Ghasempouri, "Processor vulnerability detection with the aid of assertions: Risc-v case study," in *2024 IEEE Nordic Circuits and Systems Conference (NORCAS)*. IEEE, 2024, pp. 1–6.
- [6] D. Bryan, "The iscas'85 benchmark circuits and netlist format," *North Carolina State University*, vol. 25, p. 39, 1985.
- [7] H. Rostami, M. Hosseini, A. Azarpeyvand, M. R. H. Iman, and T. Ghasempouri, "Automatic high functional coverage stimuli generation for assertion-based verification," in *2024 IEEE 30th International Symposium on On-Line Testing and Robust System Design (IOLTS)*. IEEE, 2024, pp. 1–7.
- [8] A. Jayasena and P. Mishra, "Directed test generation for hardware validation: A survey," *ACM Computing Surveys*, vol. 56, no. 5, pp. 1–36, 2024.
- [9] R. S. Chakraborty, F. Wolff, S. Paul, C. Papachristou, and S. Bhunia, "Mero: A statistical approach for hardware trojan detection," in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2009, pp. 396–410.
- [10] A. Waksman, M. Suozzo, and S. Sethumadhavan, "Fanci: identification of stealthy malicious logic using boolean functional analysis," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 697–708.
- [11] J. Zhang, F. Yuan, L. Wei, Z. Sun, and Q. Xu, "Veritrust: Verification for hardware trust," in *Proceedings of the 50th Annual Design Automation Conference*, 2013, pp. 1–8.
- [12] M. Nourian, M. Fazeli, and D. Hély, "Hardware trojan detection using an advised genetic algorithm based logic testing," *Journal of Electronic Testing*, vol. 34, pp. 461–470, 2018.
- [13] S. Chakraborty, M. S. Varenia, A. Mondal, and B. Sen, "Exploring the pso-driven test pattern generation approach for hardware trojan detection," in *2024 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*, vol. 2. IEEE, 2024, pp. 1–6.
- [14] M. Haris, D. M. S. Bhatti, and H. Nam, "A fast-convergent hyperbolic tangent pso algorithm for uavs path planning," *IEEE Open Journal of Vehicular Technology*, 2024.
- [15] S. M. Sebt, A. Patooghy, H. Beitollahi, and M. Kinsy, "Circuit enclaves susceptible to hardware trojans insertion at gate-level designs," *IET Computers & Digital Techniques*, vol. 12, no. 6, pp. 251–257, 2018.
- [16] Y. Li, S. Li, and H. Shen, "Htrans: Transformer-based method for hardware trojan detection and localization," in *2023 IEEE 32nd Asian Test Symposium (ATS)*. IEEE, 2023, pp. 1–6.